

Isotope metrics showcase — Prometheus + Grafana on minikube

A throwaway, opt-in observability stack that visualizes the three stateless Isotope reports (latency / topology / hop-distribution), the three consume-side signals, and a third **Health & ratios (operational)** row (hop failure ratio, hop completion — share reaching hop 3, consume throughput) straight from the app's Micrometer meters — no Flink, no Control Center. See [root README §3.4](#) for what each meter means.

Table of Contents

- [1.0 Architecture \(A — scrape host stages\)](#)
 - [1.1 Deploy the stack](#)
 - [1.2 Run the stages on the host \(with metrics on\)](#)
 - [1.3 Open the dashboards](#)
- [2.0 Teardown](#)
- [3.0 Troubleshooting](#)

1.0 Architecture (A — scrape host stages)

The Isotope pipeline stages run on your **host** via `./gradlew :app:run`, not in the cluster. Prometheus runs as a minikube pod and scrapes the host across the `host.minikube.internal` bridge — one host port per stage:

```

host:  gradle enrich :9410    gradle fulfill :9411    gradle ship :9412
      \                |                /
      host.minikube.internal (scraped every 5s)
      |
minikube: Prometheus :9090 → Grafana :3000

```

Port↔stage mapping lives in [10-prometheus.yaml](#); change it there if you run different stages.

1.1 Deploy the stack

```

kubectl apply -k k8s/monitoring
kubectl wait --for=condition=available deploy/prometheus deploy/grafana \
  -n monitoring --timeout=120s

```

1.2 Run the stages on the host (with metrics on)

Prereq: Kafka reachable from the host (`make kafka-pf-up`, or the CCAF env via `scripts/cc-app-run.sh`). Launch each stage in its own terminal on its mapped port. —

`Disotope.consume.from=latest` skips the replay backlog so latency/age read steady-state — see [docs/metrics.md](#).

```
# Local CP via gradle:
./gradlew :app:run -Dmetrics.prometheus.enabled=true -
Dmetrics.prometheus.port=9410 \
  -Disotope.consume.from=latest --args="enrich"
./gradlew :app:run -Dmetrics.prometheus.enabled=true -
Dmetrics.prometheus.port=9411 \
  -Disotope.consume.from=latest --args="fulfill"
./gradlew :app:run -Dmetrics.prometheus.enabled=true -
Dmetrics.prometheus.port=9412 \
  -Disotope.consume.from=latest --args="ship"

# Drive traffic so the meters move:
for i in $(seq 1 30); do ./gradlew :app:run -q --args="place burst-$i";
sleep 3; done
```

Against **Confluent Cloud** instead, use `scripts/cc-app-run.sh` (no local CP / `kafka-pf-up` needed — it pulls Cloud creds from `terraform output`, so `make cc-flink-reports-up` must have succeeded first). Leading `-D...` flags are forwarded to the app JVM; the verb is the trailing arg:

```
# CCAF via cc-app-run.sh — same metrics ports, Cloud-backed stages:
scripts/cc-app-run.sh -Dmetrics.prometheus.enabled=true -
Dmetrics.prometheus.port=9410 enrich
scripts/cc-app-run.sh -Dmetrics.prometheus.enabled=true -
Dmetrics.prometheus.port=9411 fulfill
scripts/cc-app-run.sh -Dmetrics.prometheus.enabled=true -
Dmetrics.prometheus.port=9412 ship

# Drive traffic so the meters move:
scripts/cc-app-run.sh place 'hello'
```

1.3 Open the dashboards

```
kubectl -n monitoring port-forward svc/grafana 3000:3000      # →
http://localhost:3000
kubectl -n monitoring port-forward svc/prometheus 9090:9090  # →
http://localhost:9090
```

Grafana opens straight on **"Isotope — stateless reports (Micrometer → Prometheus)"** (anonymous Admin, no login). Confirm scraping is live at Prometheus → [Status](#) → [Targets](#): each `isotope-stages` target should be **UP**. A target is **DOWN** until you start the stage on its port — that is expected.

2.0 Teardown

```
kubectl delete -k k8s/monitoring
```

3.0 Troubleshooting

- **Targets DOWN / connection refused.** The stage isn't running on that port, or `host.minikube.internal` doesn't resolve. minikube maps it to the host on the VM drivers this repo uses (`vfkit`, `kvm2`, `qemu`); if it doesn't resolve, run `minikube ssh -- getent hosts host.minikube.internal` to check, and confirm the stage binds the port with `curl -s localhost:9410/metrics | grep isotope_` on the host.
- **No data, targets UP.** The meters are lazily registered on first emission — drive some traffic (step 2). Until a stage produces/consumes, `/metrics` has no `isotope_*` series.
- **Huge latency/age values.** Backlog replay, not steady-state — use — `Disotope.consume.from=latest` ([docs/metrics.md](#)).